

SMART DOCUMENT SCANNER AND IMAGE ENHANCEMENT SYSTEM

A Python-Based Computer Vision Project Using OpenCV

PROJECT REPORT

Submitted By: Srishti Manav

Registration Number: 24BAI10314

Branch: CSE AIML

College: VIT Bhopal University

Subject: Computer Vision

Academic Year: 2026-27

2. INTRODUCTION

Document scanning is an important application of computer vision that converts physical documents into digital images. Documents captured using mobile phones or cameras may contain perspective distortion, uneven lighting, shadows, and background noise.

The **Smart Document Scanner and Image Enhancement System** is a Python-based computer vision project developed using OpenCV. The system detects document boundaries, corrects perspective distortion, and enhances the scanned image to improve its clarity and readability.

The project uses image processing techniques such as grayscale conversion, Gaussian blurring, Canny edge detection, contour detection, perspective transformation, CLAHE, and adaptive thresholding.

The system provides a simple and efficient method for converting photographed documents into scanned-style digital images.

3. PROBLEM STATEMENT

Photographs of documents often appear distorted because they are captured from different angles. Uneven lighting, shadows, and background noise can also reduce the readability of the document.

Manually cropping, straightening, and enhancing every document image can be time-consuming.

Therefore, there is a need for an automated system that can:

- Detect document boundaries.
- Correct perspective distortion.
- Improve image quality.
- Produce a clear, scanned-style document.
- Save the processed images for further use.

This project addresses these problems using Python and OpenCV-based image processing techniques.

4. FUNCTIONAL REQUIREMENTS

The functional requirements describe the operations performed by the system.

4.1 Image Input

The system accepts a document image through a specified input path.

4.2 Image Validation

The system checks whether the provided image exists and can be loaded successfully.

4.3 Grayscale Conversion

The input image is converted into grayscale to simplify image processing and edge detection.

4.4 Noise Reduction

Gaussian blur is applied to reduce unwanted noise and improve edge detection.

4.5 Edge Detection

The Canny edge detection algorithm is used to identify significant edges in the image.

4.6 Document Boundary Detection

Contours are detected and analyzed to identify a four-sided contour representing the document.

4.7 Perspective Correction

The detected document corners are used to perform a perspective transformation, producing a top-down rectangular view.

4.8 Image Enhancement

The corrected document is enhanced using contrast improvement and adaptive thresholding techniques.

4.9 Output Generation

The system saves intermediate and final output images in the specified output directory.

4.10 Error Handling

The system displays an appropriate error message when the image cannot be loaded or a document boundary cannot be detected.

5. NON-FUNCTIONAL REQUIREMENTS

5.1 Usability

The system should be easy to execute using a command-line interface.

5.2 Performance

The system should process normal document images within a reasonable amount of time.

5.3 Reliability

The application should handle invalid image paths and unsuccessful document detection.

5.4 Maintainability

The source code should be divided into modular files to simplify maintenance and debugging.

5.5 Portability

The application should run on systems that support Python and the required libraries.

5.6 Readability

The final output should improve document alignment, contrast, and visibility.

5.7 Scalability

The project should allow future additions such as OCR, PDF generation, and batch processing.

6. SYSTEM ARCHITECTURE

The system consists of the following components:

6.1 Input Layer

The user provides a document image through the command line.

6.2 Main Controller

The main.py file controls the complete processing pipeline. It accepts the input and output paths and coordinates the processing modules.

6.3 Preprocessing Module

This module performs:

- Grayscale conversion.
- Gaussian blurring.
- Canny edge detection.

6.4 Document Detection Module

This module detects contours and identifies a four-point document boundary.

6.5 Perspective Transformation Module

The detected document corners are arranged in the correct order. A perspective transformation is then applied to obtain a rectangular, top-down image.

6.6 Enhancement Module

The corrected document is enhanced using techniques such as:

- Contrast Limited Adaptive Histogram Equalization (CLAHE).
- Adaptive thresholding.

6.7 Output Layer

The processed images are saved in the output directory.

System Architecture Diagram

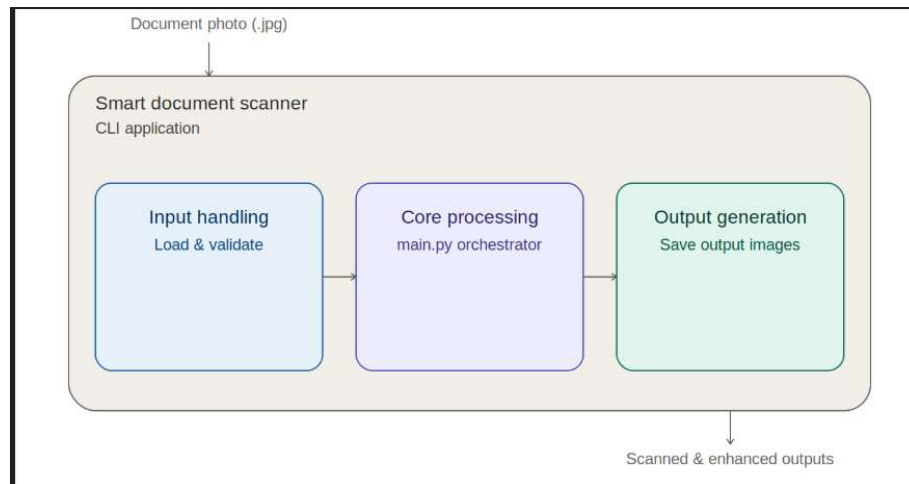


Figure 1: System architecture of the Smart Document Scanner.

7. DESIGN DIAGRAMS

7.1 Use Case Diagram

The use case diagram represents the interaction between the user and the document scanning system.

Actor

User

Use Cases

- Provide input image.
- Run document processing.
- View intermediate results.
- View scanned output.
- View enhanced output.
- Handle invalid input.

The user provides a document image and executes the application. The system processes the image and generates scanned and enhanced outputs.

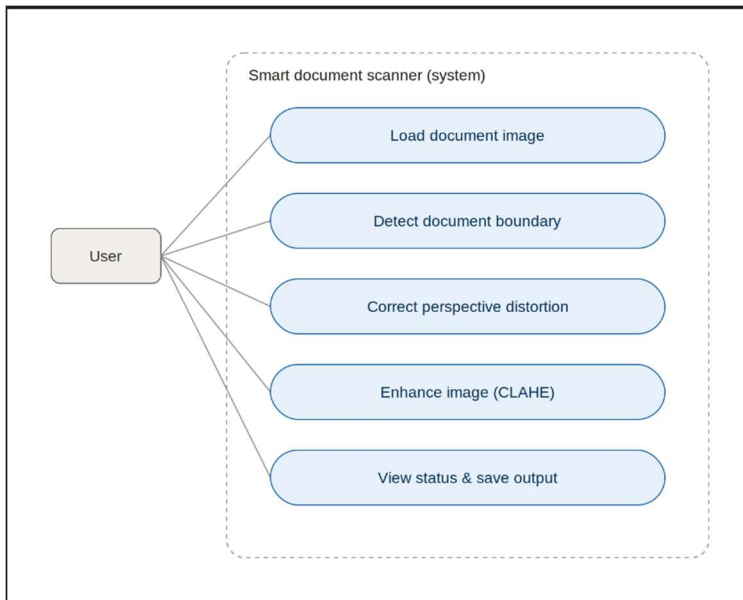


Figure 2: Use case diagram.

7.2 Workflow Diagram

The workflow of the system is as follows:

1. Start the application.
2. Load the input image.
3. Check whether the image is valid.
4. Convert the image to grayscale.
5. Apply Gaussian blur.
6. Detect edges using the Canny algorithm.
7. Find and sort contours.
8. Identify the document boundary.
9. Apply perspective transformation.
10. Enhance the corrected image.
11. Save the output images.
12. End the process.

If the input image is invalid or the document boundary cannot be detected, the system displays an error message.

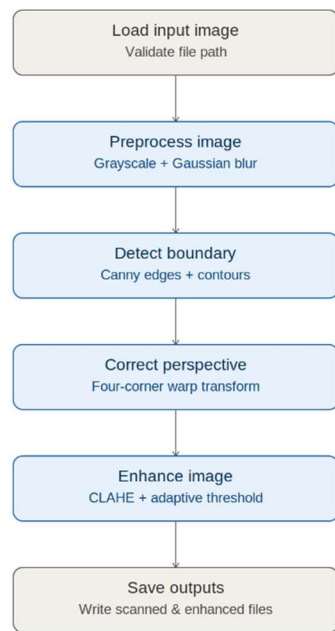


Figure 3: Workflow diagram.

7.3 Sequence Diagram

The sequence diagram describes the interaction between the user, main program, image-processing modules, and file system.

Sequence

1. The user runs the program using the command line.
2. main.py loads the input image.
3. OpenCV performs preprocessing and edge detection.
4. The document contour is identified.
5. The perspective transformation module corrects the document.
6. The enhancement module improves the image.
7. The processed images are saved.
8. The user views the output images.

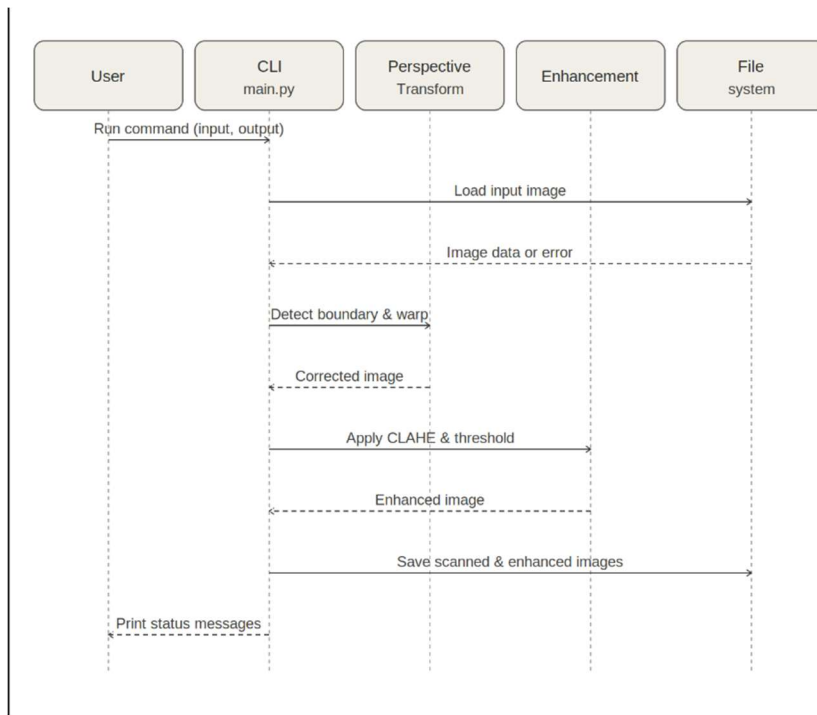


Figure 4: Sequence diagram.

7.4 Class/Component Diagram

The project is divided into the following components:

Main Controller

File: main.py

Responsibilities:

- Parse command-line arguments.
- Load the input image.
- Control the processing pipeline.
- Save output images.
- Handle errors.

Perspective Transformation Component

File: src/perspective_transform.py

Responsibilities:

- Arrange document corner points.
- Perform perspective transformation.
- Generate the corrected document image.

Image Enhancement Component

File: src/enhancement.py

Responsibilities:

- Convert the image to grayscale.
- Improve contrast using CLAHE.
- Apply adaptive thresholding.
- Generate an enhanced document image.

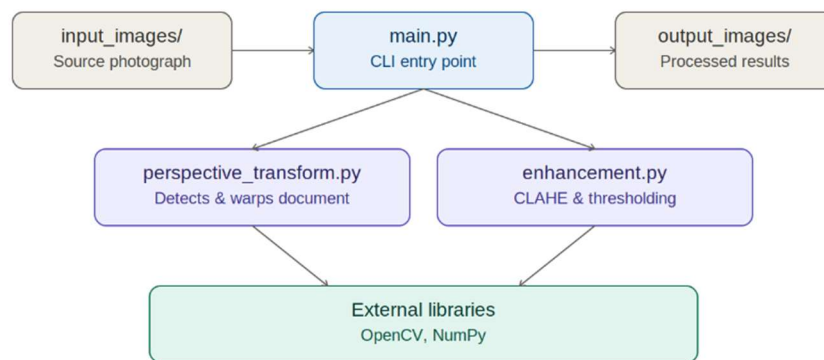


Figure 5: Class/component diagram.

8. DESIGN DECISIONS & RATIONALE

8.1 Use of Python

Python was selected because it is beginner-friendly and provides extensive libraries for computer vision and image processing.

8.2 Use of OpenCV

OpenCV provides built-in functions for:

- Image reading and writing.
- Grayscale conversion.
- Gaussian blur.
- Canny edge detection.
- Contour detection.
- Perspective transformation.
- Image enhancement.

8.3 Use of NumPy

NumPy is used to handle image data as arrays and perform numerical operations.

8.4 Use of Canny Edge Detection

Canny edge detection helps identify sharp changes in image intensity and makes it easier to locate document boundaries.

8.5 Use of Perspective Transformation

Perspective transformation converts an angled document photograph into a rectangular, top-down view.

8.6 Use of CLAHE

CLAHE improves local contrast and helps make document content more visible under uneven lighting conditions.

8.7 Modular Design

The project is divided into multiple Python files to improve code organization, maintainability, and debugging.

8.8 Local File Storage

Local folders are used because the project focuses on image processing and does not require database functionality.

9. IMPLEMENTATION DETAILS

The project is implemented using Python and OpenCV.

9.1 Main Program

The main.py file acts as the main controller. It accepts the input image path and output directory through command-line arguments.

Example command:

```
py main.py --input input_images/document.jpg --output output_images
```

The main program performs the following operations:

1. Loads the input image.
2. Converts it into grayscale.
3. Applies Gaussian blur.
4. Detects edges using Canny.
5. Finds document contours.
6. Identifies the four-point document boundary.
7. Applies perspective transformation.
8. Enhances the corrected image.

9. Saves the output images.

9.2 Perspective Transformation

The perspective transformation module identifies the four corners of the document and orders them appropriately.

The transformation produces a rectangular image that resembles a scanned document.

9.3 Image Enhancement

The enhancement module improves the quality of the corrected image.

The main techniques used are:

CLAHE

Contrast Limited Adaptive Histogram Equalization improves local image contrast.

Adaptive Thresholding

Adaptive thresholding converts the image into a binary format based on local intensity values. This can improve the visibility of handwritten or printed content.

9.4 Output Files

The project generates the following images:

File	Description
grayscale.jpg	Grayscale version of the input
edges.jpg	Canny edge detection result
document_boundary.jpg	Image with detected document boundary
scanned_document.jpg	Perspective-corrected image
enhanced_document.jpg	Enhanced document image

10. SCREENSHOTS / RESULTS

The system was tested using a photograph of a handwritten document.

10.1 Original Input Image

The original image contains a handwritten experiment page captured using a camera. The page is surrounded by a dark background and has slight perspective distortion.

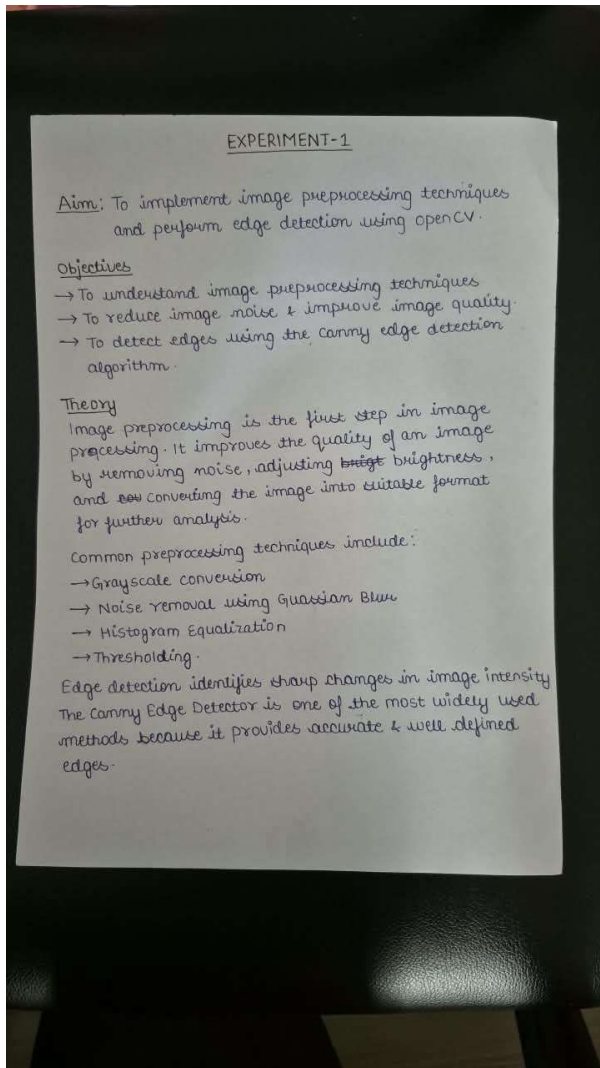


Figure 6: Original input document.

10.2 Grayscale Image

The input image is converted into grayscale. This reduces the image to intensity values and simplifies subsequent processing.

EXPERIMENT-1

Aim: To implement image preprocessing techniques and perform edge detection using openCV.

Objectives

- To understand image preprocessing techniques
- To reduce image noise & improve image quality.
- To detect edges using the Canny edge detection algorithm.

Theory

Image preprocessing is the first step in image processing. It improves the quality of an image by removing noise, adjusting ~~bright~~ brightness, and ~~con~~ converting the image into suitable format for further analysis.

Common preprocessing techniques include:

- Grayscale conversion
- Noise removal using Gaussian Blur
- Histogram Equalization
- Thresholding.

Edge detection identifies sharp changes in image intensity. The Canny Edge Detector is one of the most widely used methods because it provides accurate & well defined edges.

Figure 7: Grayscale image.

10.3 Edge Detection Output

The Canny edge detection algorithm identifies prominent edges in the image. These edges help the system locate the document boundary.

EXPERIMENT-1

Aim. To implement image preprocessing techniques and perform edge detection using OpenCV

Objectives

- To understand image preprocessing techniques
- To reduce image noise & improve image quality
- To detect edges using the Canny edge detection algorithm

Theory

Image preprocessing is the first step in image processing. It improves the quality of an image by removing noise, adjusting brightness, and then converting the image into suitable format for further analysis.

Common preprocessing techniques include:

- Grayscale conversion
- Noise removal using Gaussian Blur
- Histogram Equalization
- Thresholding

Edge detection identifies sharp changes in image intensity. The Canny Edge Detector is one of the most widely used methods because it provides accurate & well-defined edges.



Figure 8: Canny edge detection output.

10.4 Document Boundary Detection

The system detects the document boundary and highlights it using a boundary outline.

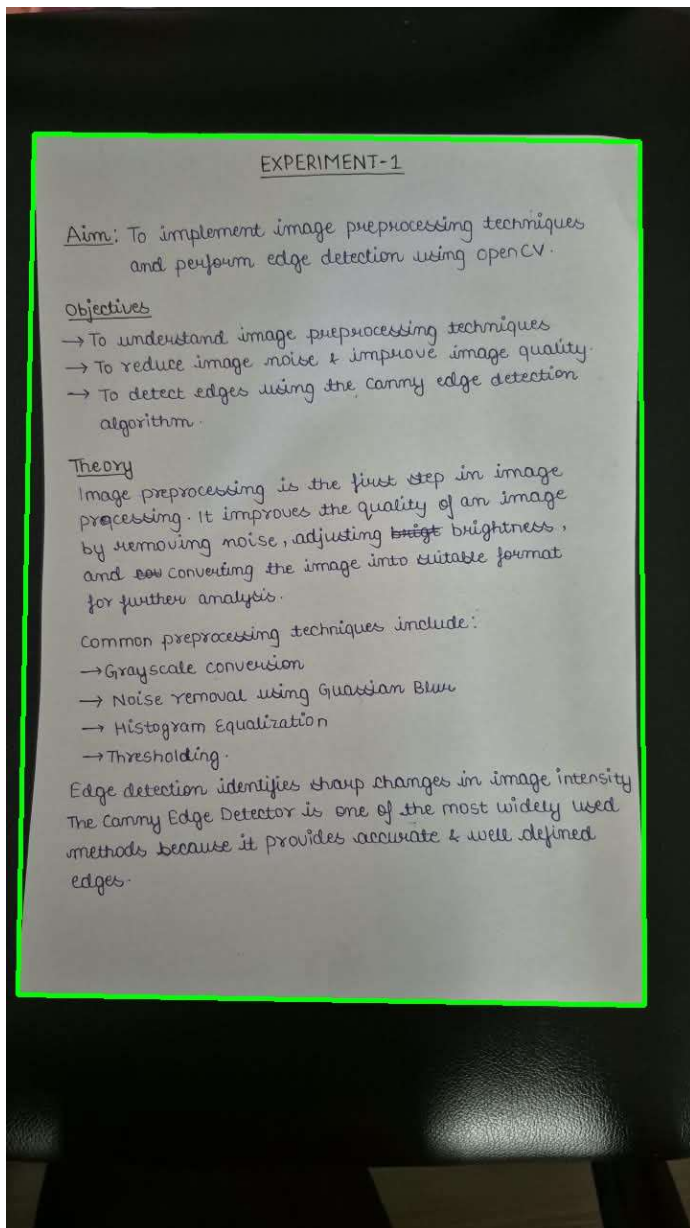


Figure 9: Detected document boundary.

10.5 Perspective-Corrected Output

The perspective transformation corrects the angled document image and generates a top-down scanned-style output.

EXPERIMENT-1

Aim: To implement image preprocessing techniques and perform edge detection using openCV.

Objectives

- To understand image preprocessing techniques
- To reduce image noise & improve image quality.
- To detect edges using the Canny edge detection algorithm.

Theory

Image preprocessing is the first step in image processing. It improves the quality of an image by removing noise, adjusting ~~brigt~~ brightness, and ~~se~~ converting the image into suitable format for further analysis.

Common preprocessing techniques include:

- Grayscale conversion
- Noise removal using Gaussian Blur
- Histogram Equalization
- Thresholding.

Edge detection identifies sharp changes in image intensity. The Canny Edge Detector is one of the most widely used methods because it provides accurate & well-defined edges.

Figure 10: Perspective-corrected document.

10.6 Enhanced Output

The final image is enhanced using contrast improvement and adaptive thresholding to improve readability.

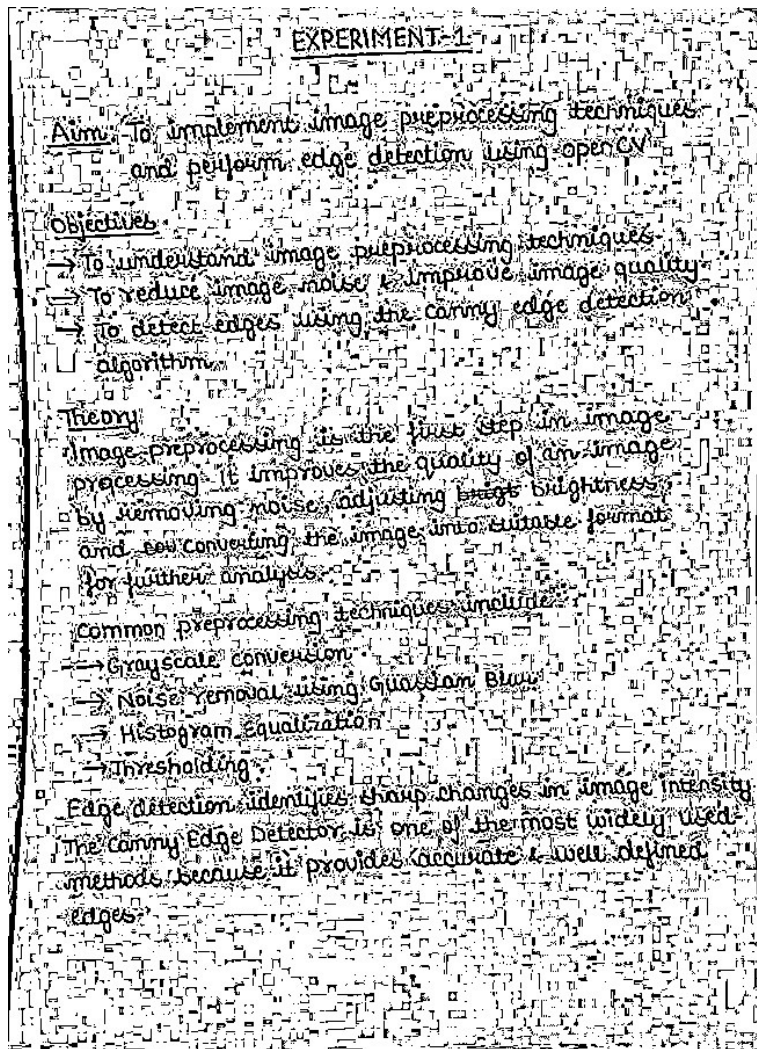


Figure 11: Enhanced document image.

10.7 Result Analysis

The system successfully demonstrates the basic stages of a document scanning pipeline:

- Image preprocessing.
- Edge detection.
- Document boundary identification.
- Perspective correction.
- Image enhancement.

The quality of the output depends on lighting, document visibility, background complexity, and the accuracy of contour detection.

11. TESTING APPROACH

Testing was performed to verify the functionality and reliability of the system.

Test Case	Input	Expected Result
Valid image	Correct image path	Image loads successfully
Invalid path	Non-existent image path	Error message is displayed
Document detection	Image containing a visible document	Document boundary is detected
Perspective correction	Angled document image	Document is transformed into a rectangular view
Enhancement	Corrected document image	Contrast and readability are improved
Output generation	Valid input and output folder	Output images are saved
Different backgrounds	Document with background objects	System attempts to detect the document contour

Testing Result

The system was tested using a sample handwritten document image. The preprocessing, edge detection, document boundary detection, perspective correction, and enhancement stages were executed successfully for the selected input.

12. CHALLENGES FACED

12.1 Image Path Errors

Initially, managing image paths and locating the input file was a challenge. This was resolved by using the correct relative paths and command-line arguments.

12.2 Python Environment Setup

Setting up the Python virtual environment and installing the required libraries required additional configuration.

12.3 Document Boundary Detection

Detecting the correct document contour can be difficult when the background contains objects or when the document edges are unclear.

12.4 Perspective Transformation

The four document corner points must be arranged correctly. Incorrect ordering can produce a distorted output.

12.5 Image Enhancement

The enhancement parameters must be selected carefully. Excessive thresholding may remove useful details or introduce noise.

12.6 Project Organization

Separating the code into modular files while ensuring that all modules work together was another implementation challenge.

13. LEARNINGS & KEY TAKEAWAYS

Through this project, the following concepts were learned:

- Fundamentals of computer vision and image processing.
- Grayscale image conversion.
- Gaussian blur and noise reduction.
- Canny edge detection.
- Contour detection and analysis.
- Perspective transformation.
- Contrast enhancement using CLAHE.
- Adaptive thresholding.
- Python virtual environment setup.
- Modular programming.
- Command-line argument handling.
- Debugging file paths and program errors.
- Organizing and uploading a project to GitHub.

The project helped develop practical knowledge of how computer vision techniques can be combined to solve real-world problems.

14. FUTURE ENHANCEMENTS

The project can be improved by adding the following features:

14.1 Optical Character Recognition

OCR can be integrated to extract text from scanned documents.

14.2 PDF Generation

The system can be enhanced to convert processed images into PDF documents.

14.3 Graphical User Interface

A GUI can be developed to allow users to select images and view results without using the command line.

14.4 Batch Processing

The system can be extended to process multiple document images automatically.

14.5 Improved Shadow Removal

Advanced image processing techniques can be used to remove shadows and uneven lighting.

14.6 Mobile Application

The system could be converted into a mobile application for real-time document scanning.

14.7 Improved Contour Detection

Machine learning or advanced contour detection techniques could improve performance when documents have complex backgrounds.

14.8 Cloud Integration

Cloud storage could be added to store and access scanned documents remotely.

15. REFERENCES

1. OpenCV Documentation
<https://docs.opencv.org/>
2. NumPy Documentation
<https://numpy.org/doc/>
3. Matplotlib Documentation
<https://matplotlib.org/stable/contents.html>
4. Python Documentation
<https://docs.python.org/3/>
5. OpenCV tutorials related to:
 - Canny edge detection
 - Contour detection
 - Perspective transformation
 - Image thresholding
6. Learning resources related to digital image processing and computer vision.